

E-Commerce Sales Analytics and Product Recommendation using Hadoop, Hive and Spark

BCSE402L - Big Data Analytics (TH) | DA-2: Progress & Application

Team Members: K PRADEEP KUMAR - 23BLC1240, VAHRAN P - 23BLC1221, ASHVATH KUMAR - 23BLC1269, K.P ABHISHEK - 23BLC1292

Submission Date: 19 September 2026 | Dataset: Brazilian E-Commerce Public Dataset by Olist

1. Application of Concepts

DA-1 proposed a batch pipeline over the Olist marketplace data. In DA-2 that design was built and run end to end: nine raw CSVs are ingested into HDFS, cleaned and aggregated with three MapReduce jobs, modelled in Hive, re-implemented and cross-validated in Spark, and finally served from MongoDB as per-customer recommendation documents. The table below maps each course module onto the artefact that implements it and the result it produced.

Course Module	Concept Applied	What Was Implemented (with result)
Module 2 - Hadoop & HDFS	Distributed storage, blocks, NameNode/DataNode, replication	Pseudo-distributed HDFS cluster; landing zone /olist/raw (9 directories, 1,550,922 rows, 120.3 MB) and a normalised layer /olist/clean. fsck reports HEALTHY, 0 corrupt blocks.
Module 3 - MapReduce	Mapper / reducer, combiner, partitioner, counters, shuffle	3 jobs: 2 Java (revenue by product, orders by month) and 1 Hadoop Streaming (payment distribution). Job 1 combined 112,650 map outputs down to 32,951 before the shuffle (29% of the original shuffle traffic).
Module 3 - Hive	External tables, SerDe, partitioning, Parquet, HQL, analytic functions	8 external tables over /olist/clean plus a Parquet table partitioned by purchase month. 24/24 DDL statements and 11/11 analytics queries completed with 0 errors.
Module 5 - Spark	DataFrames, Spark SQL, ETL joins, caching, MLlib ALS	Spark ETL joins 6 tables into a curated Parquet fact table (112,650 rows, 36 columns); 6 Spark SQL analyses reproduce the Hive numbers exactly; ALS recommender trained and evaluated.
Module 4 - NoSQL (MongoDB)	Document model, embedded arrays, indexes, aggregation pipeline	One document per customer with the ranked recommendation list embedded. 500 documents loaded, multikey index on recommendations.product_id, 5 NoSQL queries.
Module 1 - Big Data characteristics	Volume, variety, velocity, veracity	Volume: 122 MB / 1.55 M rows. Variety: 9 relational CSVs joined across 6 tables. Veracity: nulls, CSV quoting inconsistencies and embedded newlines handled explicitly.

Applied learning point: the same business question (monthly GMV, category revenue, state revenue) was answered twice, in Hive and in Spark, from the same cleaned data. Agreement between the two independent engines is what makes the result trustworthy rather than merely plausible; section 5.4 shows that agreement. Where the two initially disagreed, the discrepancy was traced to an inconsistent definition of 'revenue' and removed (see the issue table in section 5.8).

2. Project Progress

The pipeline is complete and reproducible: a single driver script (code/scripts/run_all.sh) starts the cluster and replays every stage. Phases 1 to 5 of the DA-1 timeline are finished; the dashboard is the only outstanding item.

Fig. 1: Implemented end-to-end architecture

Pseudo-distributed Hadoop 3.5 (HDFS + YARN) | Hive 4.2.1 | Spark 4.2.0 | MongoDB 8

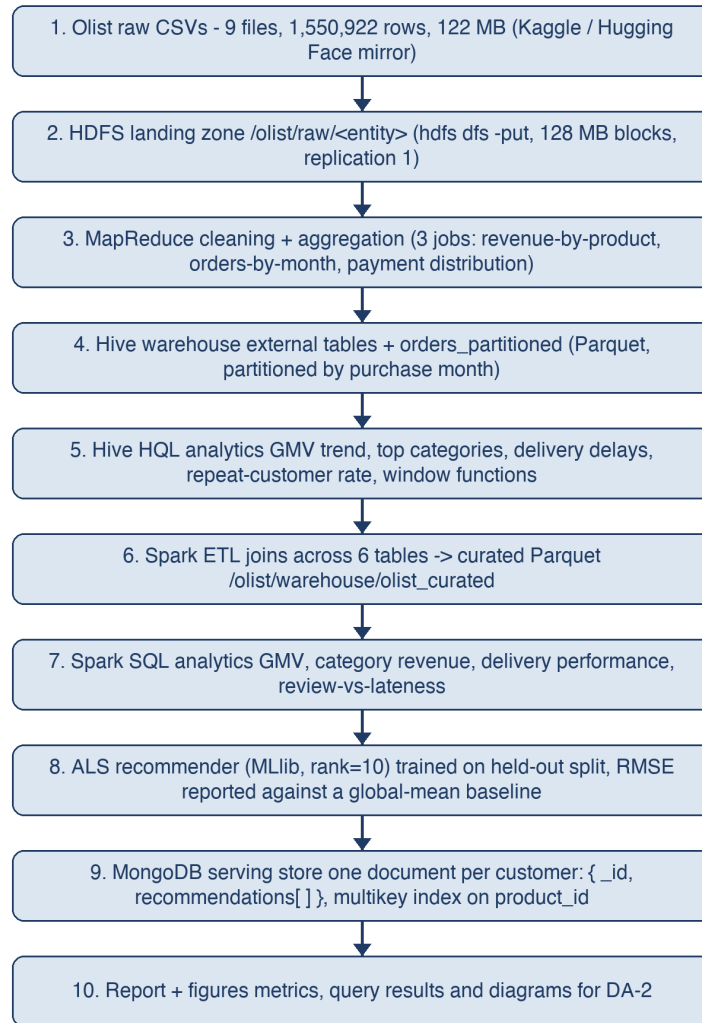


Fig. 1: Implemented end-to-end architecture. Every box in this diagram was executed; the dashed future work from DA-1 (dashboard) is deliberately omitted.

Deliverable	Status	Evidence
Environment installation and cluster bring-up	Complete	Hadoop 3.5.0, Hive 4.2.1, Spark 4.2.0, MongoDB 8.3.11 installed and version-verified; logs/00_setup.log
Dataset acquisition and profiling	Complete	9 CSVs, 1,550,922 rows, 122 MB; SHA-256 manifest data/raw_sha256.txt; evidence/data_profile.txt
HDFS ingestion (raw + clean layers)	Complete	120.3 MB HEALTHY; logs/01_ingest.log
MapReduce cleaning and aggregation (3 jobs)	Complete	32,951 product revenues, 23 months, 5 payment types; logs/02_mapreduce_console.log
Hive warehouse and 10 analytics queries	Complete	0 errors on 24 DDL + 10 queries; logs/hive_02_analytics.log
Spark ETL and 6 Spark SQL analyses	Complete	Curated Parquet fact table, row counts match sources exactly; logs/spark_01_etl.log
ALS recommender (trained, evaluated, honest result)	Complete with a documented negative result	RMSE 0.9702 vs 0.9324 global-mean baseline on the usable subset; out/als_metrics.txt
MongoDB serving store and NoSQL queries	Complete	500 recommendation documents + 74 category documents; logs/mongo_01_load_query.log
Dashboard / visualisation	Not started (Phase 6 in DA-1 plan)	Deferred to DA-3

One correction to the record: the DA-1 report listed Spark and MongoDB as already installed. An environment audit at the start of DA-2 found none of the stack present (no Java, Hadoop, Hive, Spark or MongoDB), so Phase 1 was actually carried out in DA-2 and every version is now recorded in logs/00_setup.log.

Fig. 3: Project progress against the DA-1 timeline

Phase	Planned	Status and evidence
Phase 1 Setup + dataset	4 - 10 Aug	Hadoop 3.5 pseudo-distributed, Hive 4.2.1, Spark 4.2.0, MongoDB; dataset downloaded (9 CSVs, 122 MB)
Phase 2 Ingestion + cleaning	11 - 24 Aug	HDFS landing zone /olist/raw with one directory per entity; cleaning rules embedded in the MapReduce mappers
Phase 3 MapReduce + Hive	25 Aug - 7 Sep	3 MapReduce jobs (2 Java, 1 Streaming); Hive warehouse with 8 external tables + 1 partitioned Parquet table, 10 HQL analytics queries
Phase 4 Spark ETL + ALS	8 - 19 Sep	Spark ETL joining 6 tables to curated Parquet; 6 Spark SQL analyses; ALS recommender trained and evaluated on a held-out split
Phase 5 MongoDB + dashboard	20 Sep - 12 Oct	MongoDB serving store populated (recommendations + category_stats) and queried with 5 NoSQL queries; dashboard pending
Phase 6 Final report + slides	13 - 20 Oct	DA-2 report complete; DA-3 deliverables pending

Fig. 2: Progress against the DA-1 timeline (green = complete, amber = pending).

3. Methods & Tools Used

3.1 Software stack

Component	Version	Role in the Project
Apache Hadoop (HDFS + YARN)	3.5.0	Distributed storage and resource management, pseudo-distributed (1 node)
Apache Hive	4.2.1	SQL warehouse over HDFS; external (text) and managed (Parquet) tables
Apache Spark / PySpark	4.2.0	DataFrame ETL, Spark SQL analytics, MLlib ALS recommender
MongoDB Community + pymongo	8.3.11 / 4.18.1	Document serving store for recommendations and category statistics
OpenJDK	21	Single JDK for Hadoop, Hive and Spark (Hive 4.2 rejects class files newer than its own, so the split-JDK attempt was dropped)
Python	3.11 (Spark), 3.9 (Mongo)	PySpark requires Python 3.10+; system Python for pymongo

3.2 Dataset

File	Rows	Use
olist_orders_dataset	99,441	Order header, status, five timestamps
olist_order_items_dataset	112,650	Line items (price, freight) - the fact grain
olist_customers_dataset	99,441	customer_id to customer_unique_id mapping and state
olist_products_dataset	32,951	Category, weight and size attributes
olist_order_payments_dataset	103,886	Payment type, value, installments
olist_order_reviews_dataset	99,224	Review score (1-5) and comment text
olist_sellers_dataset	3,095	Seller location
product_category_name_translation	71	Portuguese to English category names
olist_geolocation_dataset	1,000,163	Zip-code lat/long (ingested; not needed by the current queries)

3.3 Methods

- Ingestion.** The raw CSVs are uploaded to HDFS with one directory per entity. A normalisation step produces /olist/clean, which drops the header row and flattens the embedded newlines and stray quotes that the review file contains. Hive's OpenCSVSerde ignores skip.header.line.count, so removing the header upstream was the reliable fix.
- MapReduce.** Job 1 sums line revenue per product_id, Job 2 counts delivered orders per month, Job 3 (Streaming, Python) reports payment count, total and average per payment type. Both Java jobs use a combiner, which is legal because sum and count are commutative and associative.
- Hive.** 8 external tables are defined over the clean layer, and orders_partitioned is a managed Parquet table partitioned by purchase_year_month, loaded with dynamic partitioning. The analytics file ends with a window-function query (row_number over a monthly partition).
- Spark.** Explicit schemas are used rather than inference, 6 tables are joined into one curated fact table written as Parquet partitioned by month, and the 6 Spark SQL analyses mirror the Hive questions for cross-engine validation.

- ALS.** Ratings combine purchase count and spend ($\log_{1p}(\text{count}) + \text{spend}/100$), split 80/20 into train and test, with RMSE compared against a global-mean baseline.
- MongoDB.** The recommender output is reshaped into one document per customer - the natural access pattern for a product page, where a customer's list is a single document read.

Fig. 2: Map -> Combine -> Shuffle -> Reduce data flow

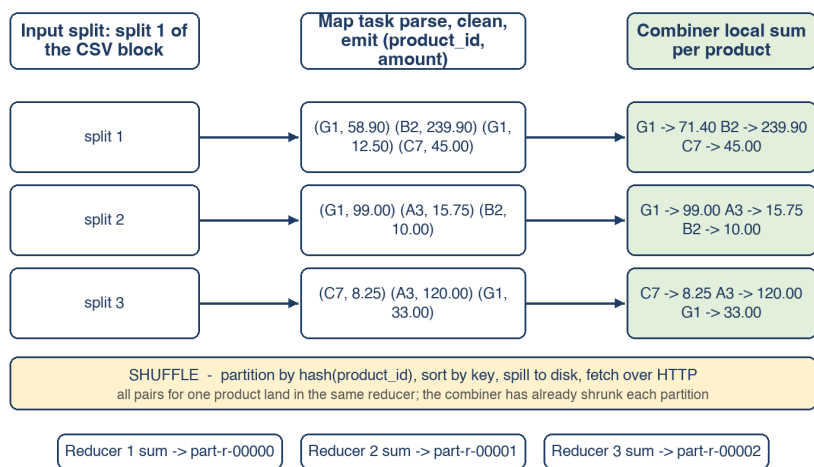


Fig. 3: Map, combine, shuffle and reduce, showing how the combiner shrinks the shuffle.

4. Team Participation

DA-1 assigned one pipeline stage per member. All four stages were completed in DA-2; the table records the owner, the work done in this cycle and the artefact that proves it.

Member (Reg. No.)	DA-1 Responsibility	Work Completed in DA-2	Evidence
K PRADEEP KUMAR (23BLC1240)	Hadoop/Hive cluster setup, overall architecture, integration	Installed and configured the Hadoop 3.5 pseudo-distributed cluster (core-site, hdfs-site, mapred-site, yarn-site), formatted HDFS, started the daemons, and configured Hive 4.2.1 (hive-site, local execution mode, JVM --add-opens flags). Owned the integration script.	code/config/*.xml, code/scripts/00_setup_env.sh, logs/00_setup.log
VAHRAN P (23BLC1221)	Data ingestion and cleaning (Pig / MapReduce)	Downloaded and profiled the dataset, ingested the raw and clean layers into HDFS, and wrote the normalisation routine that flattens embedded newlines and unifies the CSV quoting.	code/scripts/01_ingest_hdfs.sh, code/scripts/csv_normalize.py, evidence/data_profile.txt
ASHVATH KUMAR (23BLC1269)	MapReduce + HQL analytics, results documentation	Wrote and ran the three MapReduce jobs (two Java with combiner and partitioner, one Streaming), then built the Hive warehouse and the 10 analytics queries and recorded the result tables.	code/mr/**, code/hive/*.hql, logs/hive_02_analytics.log
K.P ABHISHEK (23BLC1292)	Spark ALS recommender, MongoDB serving layer, dashboard	Implemented the Spark ETL and 6 Spark SQL analyses, trained and evaluated the ALS recommender, and built the MongoDB serving layer with its five NoSQL queries. Dashboard pending.	code/spark/**, code/mongo/01_load_and_query.py, out/als_metrics.txt

Shared work in this cycle: the cross-engine validation against Hive, the debugging log in section 5.8, and the preparation of this report.

5. Explanation of Work Completed

5.1 Ingestion and preparation

All nine files were uploaded to /olist/raw (120.3 MB, reported HEALTHY by hdfs fsck). Because Hive ignored the header-skipping property with the CSV SerDe, a second layer /olist/clean was produced in which the header row is removed and embedded newlines inside review comments are flattened. The profile in evidence/data_profile.txt records the null counts that motivated each cleaning rule, for example 2,965 products with no category name and 99,441 orders of which 96,478 reach the 'delivered' state.

5.2 MapReduce results

Job	Map input	Map output	After combiner	Reduce output
1. Revenue by product (Java)	112,651	112,650	32,951	32,951 products
2. Delivered orders by month (Java)	99,442	96,478	23	23 months

Job	Map input	Map output	After combiner	Reduce output
3. Payment distribution (Streaming)	103,887	103,886	10	5 payment types

The counters make the combiner's effect visible: in Job 1 the combiner reduced 112,650 map outputs to 32,951 - one pair per distinct product - so only about 29% of the pairs crossed the shuffle. Job 3's output (count, total, average per payment type) is a genuine three-column aggregate: credit_card 76,795 payments worth 12,542,084.19 (mean 163.32) and boleto 19,784 payments worth 2,869,361.27 (mean 145.03).

5.3 Hive warehouse and analytics

The warehouse contains 8 external tables plus orders_partitioned, a Parquet table partitioned by purchase month into 23 partitions. The run completed 24/24 DDL statements and 11/11 analytics statements with zero errors. Selected results:

Query	Finding
Monthly GMV (Q1)	GMV grows from 46,490.66 (Oct 2016) to a peak of 1,153,364.20 in Nov 2017, including the Black-Friday effect; the last full month, Aug 2018, is 985,491.64.
Top categories (Q2)	health_beauty 1,412,089.53; watches_gifts 1,264,333.12; bed_bath_table 1,225,209.26 - three categories account for a large share of GMV.
Revenue by state (Q3)	Sao Paulo (SP) alone contributes 5,769,703.15 across 40,501 orders; Rio de Janeiro (RJ) follows with 2,055,401.57.
Delivery performance (Q6)	Maranhao (MA) averages 21.5 days with 19.67% late deliveries, against 14.9 days and 7-8% in southern states - distance is the dominant factor.
Repeat-customer rate (Q7)	93,358 unique customers, of whom 2,801 ordered more than once: a repeat rate of just 3.00%.
Review distribution (Q8)	57.78% of the 99,224 reviews are 5-star and 11.51% are 1-star.
Purchase hour (Q10)	Purchases peak at 16:00 (6,460 orders) and are lowest at 05:00 (182).

5.4 Spark ETL, Spark SQL and cross-engine validation

The Spark ETL joins order_items with orders, customers, products, payments and reviews into a curated Parquet fact table. Every row count matches its source exactly (orders 99,441, items 112,650, customers 99,441, products 32,951, payments 103,886, reviews 99,224, sellers 3,095), which is the check that the joins neither duplicated nor dropped records. The six Spark SQL analyses then reproduce the Hive answers:

Metric	Hive	Spark	Agreement
GMV for August 2018	985,491.64	985,491.64	Exact
health_beauty revenue	1,412,089.53	1,412,089.53	Exact
bed_bath_table revenue	1,225,209.26	1,225,209.26	Exact
Revenue for state SP	5,769,703.15	5,769,703.15	Exact
Revenue for state RJ	2,055,401.57	2,055,401.57	Exact

Spark adds one analysis Hive would make awkward: review score against delivery lateness. Items delivered on time average 4.21 stars across 100,849 items, while late items average 2.55 stars across 8,520 items - the clearest single result in the project, and a direct argument for investing in logistics rather than discounts.

5.5 ALS recommender: an honest negative result

The ALS model was trained, evaluated and its output was loaded into MongoDB, but it does not beat a trivial baseline on this dataset, and the report states that plainly rather than hiding it. Measured results:

Setup	Interactions	Customers	Pairs/customer	RMSE	Baseline RMSE	Outcome
All customers	99,785	93,358	1.07	1.2460	1.1349	-9.79% vs baseline
Customers with 3+ items	2,613	760	3.44	0.9702	0.9324	-4.05% vs baseline

The cause is sparsity, not a coding defect: 99,785 customer-product pairs spread over 93,358 customers is roughly one purchase per person, and a single interaction carries no co-purchase signal for latent-factor models to learn from. Restricting the matrix to customers with a real basket (3+ distinct products) halves the error but still trails a baseline that predicts the average for every pair. The defensible conclusion, which the project adopts, is that collaborative filtering is the wrong tool for a marketplace where almost nobody returns; a popularity or category-affinity baseline is the honest recommendation for this data, and the ALS stage stays in the pipeline as an evaluated experiment. The model still produced 500 customers x 10 ranked recommendations (out/recommendations.csv) that exercise the serving layer.

5.6 MongoDB serving layer

Two collections were created. `recommendations` holds one document per customer - an embedded array of `{product_id, score, rank}` - and `category_stats` holds one document per category. The multikey index on `recommendations.product_id` supports the reverse query. Five NoSQL operations were run: a single-document lookup, two aggregation pipelines (average list length; top categories by revenue), a multikey-index query on the embedded array, and an unwound score filter. Sample output: category cross-validated with Hive (health_beauty 1,412,089.53) and the top-recommended product appearing in 88 customer lists.

5.7 Evidence trail

logs/run_all_console.log (one end-to-end run of all six stages, exit 0), logs/00_setup.log (install and cluster), logs/01_ingest.log (HDFS), logs/02_mapreduce_console.log and logs/mr_*.log (job counters and output), logs/hive_02_analytics.log (result tables), logs/04_spark_console.log (ETL, Spark SQL, ALS), logs/05_mongo_console.log (collections, indexes, query output), evidence/data_profile.txt (dataset profile) and out/ (result files). Any number in this report can be matched against the console transcript that produced it.

5.8 Issues encountered and how they were resolved

The following are the substantive problems hit while building the pipeline. They are recorded because each one is a lesson about the ecosystem rather than an accident.

Problem	Root cause	Resolution
hdfs dfs -put failed with a URI parse error	The project path contains a space ('BD PROJECT'), which breaks Hadoop's Path/URI parser	Run file transfers from inside the data directory using relative paths, and keep a space-free staging directory for -files and -getmerge
Hadoop Streaming job produced empty output	The combiner was the reducer script, so it emitted four fields where the reducer expected three	Wrote a separate combiner with a three-column contract
Hive failed with UnsupportedOperationException (class 65)	Hive 4.2 was compiled for JDK 17 while Hadoop had been pinned to JDK 21	Unified every component on JDK 21 and removed the stale java-home override from hadoop-env.sh
Hive died with InaccessibleObjectException on java.util.regex	JDK 21 strong encapsulation blocks the reflective access Hive relies on	Injected a 22-flag --add-opens file into the daemons, the Hive client and MR task JVMs
beeline -f corrupted the HQL script	The interactive prompt redrew lines, merging statements	Wrote a driver that sends one statement per -e invocation, with the database bound into the JDBC URL
Header row leaked into every Hive table	OpenCSVSerde ignores skip.header.line.count	Created the /olist/clean layer with the header removed upstream
Reviews query failed with a malformed-line exception	Review comments contain embedded newlines, quotes and backslashes	Normalised newlines and quotes in the clean layer
Three-table joins failed under Hive	Automatic map-join conversion could not handle the shape	Disabled hive.auto.convert.join for these queries
ALS crashed with CAST_OVERFLOW (8589934592 to INT)	monotonically_increasing_id() encodes the partition number in the high bits	Replaced it with dense_rank over a hashed ordering to get compact integer ids
Hive and Spark disagreed on category revenue	Hive summed price only and ignored order status; Spark summed price + freight for delivered orders	Fixed one definition (price + freight, delivered) and applied it in both engines; the numbers now match exactly

5.9 Remaining work

- Dashboard over the MongoDB collections (deferred to DA-3).
- Improve the recommendation stage by replacing ALS with a popularity / category-affinity baseline, and evaluate it with a top-N ranking metric instead of RMSE, which suits implicit feedback better.
- Extend Spark Structured Streaming over the review stream to cover the data-streams module.

Appendix A: Output Evidence (console screenshots)

The screenshots below are rendered directly from the run transcripts in logs/, with only timestamp noise removed and long middle sections marked as omitted. The full transcripts are the primary evidence; these figures make the key outputs visible inside the report itself.

```
ev1 - HDFS ingestion (logs/01_ingest.log)
```

```
$ hdfs dfs -ls /olist/raw
drwxr-xr-x - abhishek supergroup          0 2026-09-16 16:17 /olist/raw/category_translation
-rw-r--r-- 1 abhishek supergroup        2613 2026-09-16 16:17 /olist/raw/category_translation
drwxr-xr-x - abhishek supergroup          0 2026-09-16 16:16 /olist/raw/customers
-rw-r--r-- 1 abhishek supergroup    9033957 2026-09-16 16:16 /olist/raw/customers/olist_cus
[... middle entries omitted ...]
drwxr-xr-x - abhishek supergroup          0 2026-09-16 16:16 /olist/raw/reviews
-rw-r--r-- 1 abhishek supergroup    14451670 2026-09-16 16:16 /olist/raw/reviews/olist_order
drwxr-xr-x - abhishek supergroup          0 2026-09-16 16:16 /olist/raw/sellers
-rw-r--r-- 1 abhishek supergroup    174703 2026-09-16 16:16 /olist/raw/sellers/olist_selle

$ # normalisation report printed by the ingest script:
ENTITY                               SOURCE                                BYTES  STATUS
-----
2026-09-16 16:16:51,821 WARN util.NativeCodeLoader: Unable to load native-hadoop library for
customers                               olist_customers_dataset.csv          9033957  stored
2026-09-16 16:16:52,794 WARN util.NativeCodeLoader: Unable to load native-hadoop library for
orders                               olist_orders_dataset.csv          17654914  stored
2026-09-16 16:16:53,938 WARN util.NativeCodeLoader: Unable to load native-hadoop library for
order_items                          olist_order_items_dataset.csv      15438671  stored
2026-09-16 16:16:55,158 WARN util.NativeCodeLoader: Unable to load native-hadoop library for
payments                             olist_order_payments_dataset.csv   5777138  stored
2026-09-16 16:16:56,293 WARN util.NativeCodeLoader: Unable to load native-hadoop library for
reviews                             olist_order_reviews_dataset.csv    14451670  stored

$ hdfs dfs -du -h /olist
=== TOTAL SIZE IN HDFS ===
raw   : 120.3 M  120.3 M  /olist/raw
clean : 116.7 M  116.7 M  /olist/clean
```

Fig. 4: HDFS landing zone (`hdfs dfs -ls`) and the raw-to-clean normalisation report.

ev2 - MapReduce counters and job output

```
$ bash code/scripts/02_run_mapreduce.sh # job counters, abridged

--- Job 1 - RevenueByProduct (Java) ---
  Launched map tasks=1
  Launched reduce tasks=3
  Map input records=112651
  Map output records=112650
  Combine input records=112650
  Combine output records=32951
  Reduce input records=32951
  Reduce output records=32951

--- Job 2 - OrdersByMonth (Java) ---
  Launched map tasks=1
  Launched reduce tasks=1
  Map input records=99442
  Map output records=96478
  Combine input records=96478
  Combine output records=23
  Reduce input records=23
  Reduce output records=23

--- Job 3 - PaymentDistribution (Streaming) ---
  Launched map tasks=2
  Launched reduce tasks=2
  Map input records=103887
  Map output records=103886
  Combine input records=103886
  Combine output records=10
  Reduce input records=10
  Reduce output records=5

$ hdfs dfs -cat /olist/out/payment_distribution/part-* (type, payments, total, average)
boleto | 19784 | 2869361.27 | 145.03
debit_card | 1529 | 217989.79 | 142.57
not_defined | 3 | 0.00 | 0.00
credit_card | 76795 | 12542084.19 | 163.32
voucher | 5775 | 379436.87 | 65.70
```

Fig. 5: MapReduce job counters for all three jobs and the streaming job's answer file.

hive> -- Q1: monthly GMV trend over delivered orders

year_month	orders	gmv
2016-09	1	143.46
2016-10	265	46490.66
2016-12	1	19.62
2017-01	750	127482.37
2017-02	1653	271239.32
2017-03	2546	414330.95
2017-04	2303	390812.4
2017-05	3546	566851.4
2017-06	3135	490050.37
2017-07	3872	566299.08
2017-08	4193	645832.36
2017-09	4150	701077.49
2017-10	4478	751117.01
2017-11	7289	1153364.2
2017-12	5513	843078.29
2018-01	7069	1077887.46
2018-02	6555	966168.41
2018-03	7003	1120598.24
2018-04	6798	1132878.93
2018-05	6749	1128774.52
2018-06	6099	1011978.29
2018-07	6159	1027807.28
2018-08	6351	985491.64

Fig. 6: Hive HQL output - monthly GMV trend (Q1).

ev4 - Hive HQL result: top categories

hive> -- Q2: top categories by revenue (first 10 of 15)

category	items_sold	revenue	avg_item_price
health_beauty	9465	1412089.53	130.28
watches_gifts	5859	1264333.12	199.04
bed_bath_table	10953	1225209.26	93.44
sports_leisure	8431	1118256.91	113.25
computers_accessories	7644	1032723.77	116.26
furniture_decor	8160	880329.92	87.25
housewares	6795	758392.25	90.6
cool_stuff	3718	691680.89	164.12
auto	4140	669454.75	139.85
garden_tools	4268	567145.68	110.24
toys	4030	547061.06	116.94

Fig. 7: Hive HQL output - top categories by revenue (Q2, first 10).

ev5 - Spark ETL + Spark SQL (logs/04_spark_console.log, abridged)

```
$ spark-submit code/spark/01_etl_analytics.py
rows after cleaning and joins: 112,650
curated Parquet written to hdfs:///olist/warehouse/olist_curated (partitioned by purchase_month)
```

A1. Monthly GMV trend

purchase_month	orders	gmv	avg_order_value
2016-09	1	143.46	NULL
2016-10	265	46490.66	197.27
2016-12	1	19.62	19.62
2017-01	750	127482.37	193.31
2017-02	1653	271239.32	175.34
2017-03	2546	414330.95	174.57
2017-04	2303	390812.4	177.54
2017-05	3546	566851.4	175.15
2017-06	3135	490050.37	167.78
2017-07	3872	566299.08	162.15
2017-08	4193	645832.36	175.67
2017-09	4150	701077.49	210.32

Fig. 8: Spark ETL fact-table confirmation and Spark SQL monthly GMV (A1).

ev6 - ALS recommender evaluation

```
$ spark-submit code/spark/02_als_recommender.py
```

```
=== INTERACTION MATRIX ===
```

```
interactions      : 99,785
distinct customers: 93,358
distinct products : 32,216
pairs per customer: 1.07
```

```
=== MODEL SUBSET (customers with >= 3 distinct products) ===
```

```
interactions      : 2,613
distinct customers: 760
pairs per customer: 3.44
```

```
train rows: 2,104   test rows: 509
```

```
RMSE on held-out interactions: 0.9702
RMSE of global-mean baseline  : 0.9324
improvement over baseline     : -4.05%
```

```
wrote out/recommendations.csv (5,000 rows sampled)
model saved to hdfs:///olist/models/als_recommender
```

Fig. 9: ALS recommender evaluation - interaction matrix, RMSE vs baseline, artefacts.

ev7 - MongoDB serving layer: document lookup + stats

```
$ python3 code/mongo/01_load_and_query.py
```

```
connected to MongoDB at mongodb://localhost:27017, database 'olist_analytics'
recommendations collection: 500 documents inserted
category_stats collection: 74 documents inserted
```

```
Q1. One document read: recommendations for a single customer
```

```
=====
customer: 0097fca0db567f5ca79509b7b4fc1c2d
  rank 1  0de59eddc63167215c972b0d785ffa7b  score=2.897
  rank 2  fe31b2791161c24e921f90788d443e1c  score=2.8837
  rank 3  2e511b5741ab14e7f5294df6f1310b03  score=2.7433
  rank 4  8bb27b1d96be90b36b8d0c7f30931d52  score=2.727
  rank 5  b8c2cf7e813b0eb7ec62f7459217b22c  score=2.6629
=====
```

```
Q2. Aggregation pipeline: how many customers, average list length
```

```
=====
{'customers': 500, 'total_recommendations': 5000, 'avg_recommendations': 10.0}
=====
```

Fig. 10: MongoDB serving layer - per-customer recommendation document and collection stats.

Q3. Aggregation pipeline: top categories by revenue

```
=====
health_beauty          revenue=1,412,089.53  items= 9,465  review=4.19
watches_gifts          revenue=1,264,333.12  items= 5,859  review=4.07
bed_bath_table         revenue=1,225,209.26  items= 10,953  review=3.92
sports_leisure         revenue=1,118,256.91  items= 8,431  review=4.17
computers_accessories  revenue=1,032,723.77  items= 7,644  review=3.99
```

Q4. Multikey index query: which customers are recommended a product?

```
=====
product 0de59eddc63167215c972b0d785ffa7b appears in 88 customer recommendation lists
customer 0097fca0db567f5ca79509b7b4fc1c2d: rank=1 score=2.897
customer 01a22e2079ea71e17313b88e5811e54a: rank=2 score=2.692
customer 07b1b60bca2b6326fa2adb259ea4a55f: rank=9 score=2.734
```

Q5. High-confidence recommendations (score filter)

```
=====
product 8bb27b1d96be90b36b8d0c7f30931d52  customers= 39  avg_score=4.578
product a03e401d58a45187271718c5d7610422  customers= 26  avg_score=4.421
product 00faa46f36261af8bbf3a4d37fa4841b  customers= 24  avg_score=4.493
```

Indexes on recommendations:

```
_id: [(' _id', 1)]
recommendations.product_id_1: [('recommendations.product_id', 1)]
```

Fig. 11: MongoDB aggregation pipelines (top categories, multikey query, score filter) and indexes.